

RAGLoom

Technical evaluation brief · for a product / engineering PM

A visual RAG pipeline editor for local, on-device product Q&A.

Drop in product spec documents, ask questions in natural language, and get grounded answers — generated entirely on-device, with no cloud dependency. Built in Python, FastAPI, React and ChromaDB on Ollama. **No LangChain.** This brief is written to be **evaluated**, not just read: what each part does, why it was built that way, the limits, and how you can verify the claims yourself.

Jun-Fu (Jeff) Liu · portfolio.jeffliu.com · github.com/Tsoleou/RAGLoom · Product Brief · v3

THE PROBLEM

How do you run a trustworthy product-Q&A demo on a laptop, with no internet and a small model — without it inventing specs or recommending a competitor?

A booth wants a live product assistant, and on-device inference doubles as proof of AI-PC performance. But local-only pushes you toward a small model (default `gemma3:4b`, swappable) — and small models hallucinate, misread numbers, and can't be trusted to follow instructions on what matters at a booth: never quote a price, name a competitor, or recommend a 1.8 kg laptop for a "under 1 kg" request.

THE APPROACH

RAGLoom moves every high-stakes decision **out of the prompt and into code**. Four deterministic gates short-circuit a query *before* it reaches the generator; the LLM is left to do the one thing it's good at — phrasing a grounded answer.

Why it's evaluable: safety lives in code you can read and test, not a prompt you hope the model obeys. Each gate is a unit you can trigger and verify on its own.

THE SAFETY LAYER — POLICY IN CODE, NOT IN THE PROMPT

- **Guardrail**

Keyword block list (e.g. competitor brands) → canned refusal. *Deterministic: a banned term always refuses, on every run.*

- **PriceGuard**

Detects price intent (price / MSRP / 多少錢...) and refuses *before* retrieval. *So a pricing question never reaches the model to guess a number.*

- **ScopeGate**

Semantic on/off-topic check. *Catches off-topic questions smuggled behind an on-topic product name ("my gran loved StarForge — what's the weather?"), which a plain similarity score waves through.*

- **ConstraintFilter**

Numeric spec gate. Resolves each candidate's real spec and drops violators — so a 1.8 kg laptop is never recommended for a "under 1 kg" request, instead of trusting the model to do the < comparison.

THE PRODUCT

WHO IT'S FOR & WHAT YOU'D ACTUALLY TOUCH

End users / booth visitors

A Chat UI: ask a question, get a formatted, grounded answer with a retrieval-details panel and (in chatbot mode) an animated avatar. Replies come back in the visitor's language.

Builders & operators — you

A Node Editor to assemble, rewire and A/B the pipeline, plus an in-editor batch eval harness to measure retrieval quality *before* you ship a graph variant. Configuration is the graph + the KB documents — not a code fork.

CONVERSATIONAL INQUIRY FLOW — A BOOTH ASSISTANT, NOT A SEARCH BOX

`IntentRouter` classifies each visitor turn (one small LLM call) into one of four inquiry intents, re-detected *every turn* so a topic-hopping visitor is followed immediately. `DialogueFlow` then runs the matching multi-stage script:

SPEC	Answer the asked spec from the KB
RECOMMEND	Elicit needs → Recommend 1–2 models with a reason
COMPARE	Confirm which models → List key differences + a fit tip
SUITABILITY	Clarify the use case → Judge fit; suggest an alternative if not

Guard-aware state. Stage and intent only advance on a *committed* turn — one where the generator actually produced an answer. A guard short-circuit freezes the funnel, so refusals and off-topic turns never pollute the conversation's stage or history. The single advance gate (`decide_advance`) is the one place designed to swap LLM judgement for deterministic slot logic — *i.e. you can harden a script from "model decides" to "rules decide" in one spot, without touching the rest.*

CAPABILITIES — WHAT'S IN THE BOX

● Visual node editor

27 node types across ingest / query / eval. Real-time per-node execution status over WebSocket; LLM-invoking nodes carry a distinct background tint, so the compute-heavy steps are obvious at a glance.

● Dialogue-aware routing

IntentRouter + DialogueFlow turn the pipeline into a guided booth conversation — four inquiry scripts with guard-frozen state, shipped as the active `booth_inquiry` profile.

● Metadata-filtered retrieval

Docs tagged with a `product_id` at ingest. A ProductSelector routes point-queries to a single product (rule mode = zero LLM latency); comparisons fall through to broad search.

● Retrieval Judge

An LLM rerank pass drops chunks that look similar but actually answer the *opposite* (negation / polarity) — *something pure vector similarity can't tell apart*. Degrades to keep-all on error.

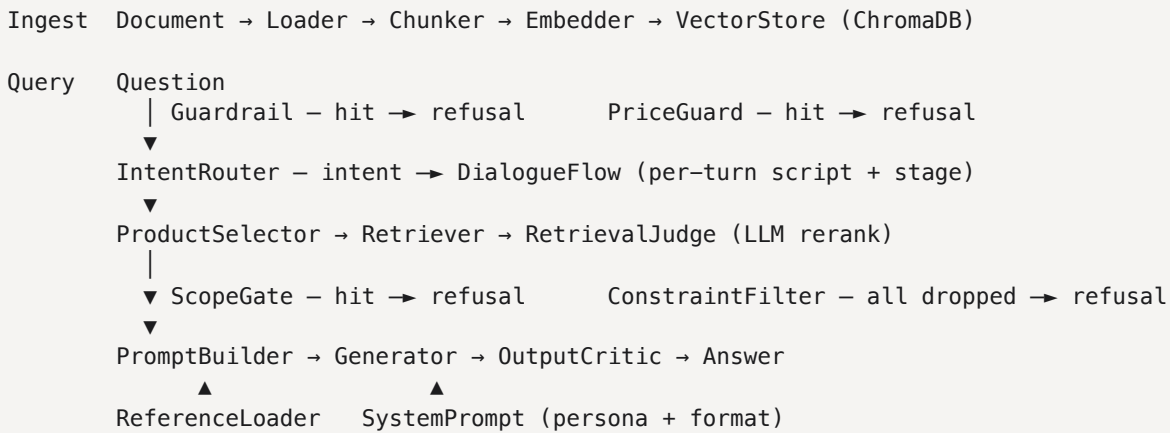
● Always-on reference data

A static comparison table injected into every prompt, guaranteeing broad coverage for comparison queries independent of vector retrieval.

● Persona presets & structured output

professional / chatbot registers. Chatbot mode emits grammar-constrained JSON `{reply, emotion}` — *the shape is enforced by the decoder, not requested in a prompt, and rendered as a themeable avatar.*

ARCHITECTURE



Read left-to-right: any gate that fires exits early to a canned refusal — the generator only runs on a query that passed every gate.

HOW YOU'D EVALUATE IT YOURSELF

Nothing below asks you to take quality on trust — each is something you can measure before a show:

- **Golden-set eval with LLM-as-judge** — a curated regression set, audited by a second LLM that returns explicit `hallucinated_claims` lists. Gates on the binary signal (claims empty / non-empty), not noisy float scores, so same-commit reruns don't flicker pass/fail. *This is the answer-quality check.*
- **In-editor batch eval** — sweep the golden set through your graph and read macro averages, per-category breakdown and worst-K. *This measures retrieval quality (did we fetch the right context?) — deterministic and retrieval-only, separate from answer wording.* One click exports a self-contained HTML report of the run.
- **Hardened local API** — every endpoint requires an auto-generated local token, CORS is locked to the local origin, file-path params are allowlisted, and batch eval is bounded. A stray browser tab can't drive your local pipeline.

SCOPE & LIMITS — WHAT IT IS *NOT*

Stated up front, because an honest boundary is what makes the rest evaluable:

- **Model quality is a lever you control — and it's swappable.** Each LLM node's model is a per-node setting: `gemma3:4b` is the default, and you can point any node at a larger local model where the booth hardware allows. The design *target* is staying trustworthy on a small model, not being locked to one; answer fluency tracks whatever model you pick.
- **Two different quality questions, two different tools.** Batch eval measures retrieval; it does *not* score answer wording. Answer hallucination is caught by the separate golden-set LLM-judge. Don't read a green batch run as "answers are correct."
- **Garbage in, garbage out on the KB.** Retrieval quality depends on curated, `product_id` – tagged spec documents. Poorly written or mistagged docs surface as low retrieval confidence, not silent wrong answers — but they still need fixing.
- **Single-machine, session-per-visitor.** Built for one booth conversation at a time, not a high-concurrency multi-tenant server.
- **Tuned for Chinese & English.** Guardrails, aliases and personas are validated for zh/en; other languages will answer but aren't regression-covered.

TECH STACK

Python · FastAPI · **React** · Vite · TypeScript · ChromaDB · **Ollama** (default `gemma3:4b`, per-node swappable) · `nomic-embed-text` · WebSocket · **No LangChain**

AT A GLANCE

27

NODE TYPES

4

CODE-LEVEL GATES

0

CLOUD CALLS